



Universidad Tecnológica Nacional
Facultad Regional Córdoba

Trabajo práctico N°2: programación de la placa BluePill en BareMetal

Ejercicio Práctico N°2: Conversión Analógica-Digital (ADC)

Técnicas Digitales II

4R2

Gastón Grasso 401892

Franco Palombo 401910

Angelo Prieto 401012

Última actualización: 8 de junio de 2026

Índice

1	Introducción	2
2	Desarrollo	3
2.1	Archivo de Inicialización y Script de Enlazado	3
2.2	Código Fuente Principal en C	3
2.2.1	Mapeo de Registros Adicionales	6
2.2.2	Configuración de Relojes y Prescaler del ADC	6
2.2.3	Configuración de los Puertos GPIO	7
2.2.4	Inicialización y Calibración del ADC	8
2.2.5	Lógica de Conversión por Muestreo (Polling)	8
2.2.6	Procesamiento de Datos y Visualización	8
2.3	Relación entre Archivos de Desarrollo	9
3	Conclusiones	10
4	Fuentes	11

1 Introducción

Este informe documenta el desarrollo y la resolución del Ejercicio Práctico N° 2 del Trabajo Práctico N° 2. Las actividades propuestas en esta etapa tienen como objetivo principal estudiar y comprender la configuración y puesta en marcha del convertidor analógico-digital (ADC1) en el microcontrolador STM32F103C8T6 bajo un esquema de programación Bare Metal puro.

El propósito general de este ejercicio es implementar la adquisición de señales analógicas y su posterior procesamiento digital. Para ello, se busca comprender el funcionamiento del periférico de conversión, la configuración de canales múltiples, el establecimiento de tiempos de muestreo y factores de escala, y la representación de las mediciones obtenidas, la cual se logra mediante el uso de 5 LEDs. A su vez, se integra el control de eventos por hardware para alternar dinámicamente entre las fuentes de señal de entrada, vinculando las técnicas de interrupciones externas estudiadas en el ejercicio anterior con los métodos de muestreo de este periférico.

2 Desarrollo

Como se mencionó en la introducción de este informe, el desarrollo de este ejercicio práctico consiste en la implementación de un sistema de adquisición analógico con visualización adaptativa y control de selección por hardware. Para ello, se conecta un potenciómetro en el puerto PA0 configurado como entrada analógica al periférico ADC1. Por otro lado, los puertos PA1 al PA5 se disponen como salidas digitales para controlar una barra de cinco leds externos.

El comportamiento del sistema está gobernado por un botón pulsador conectado al puerto PC15, el cual está configurado para disparar una interrupción externa (EXTI). Esta interrupción conmuta dinámicamente entre dos modos de visualización en la barra de leds:

- La tensión medida en el potenciómetro externo mediante el canal 0 del ADC.
- La temperatura medida por el sensor interno del microcontrolador (canal 16 del ADC1), calculada mediante expresiones aritméticas de precisión entera.

Adicionalmente, se mantiene un led testigo interno parpadeando en el puerto PC13 como indicador de que el bucle principal se ejecuta sin bloqueos. A continuación, se detalla la configuración a nivel de registros y la estructura del firmware implementado.

2.1 Archivo de Inicialización y Script de Enlazado

Tanto el archivo de arranque en ensamblador `crt.s` como el script de enlazado `linker.ld` no sufrieron ningún tipo de modificación respecto a la implementación detallada en el Ejercicio Práctico N° 1. Por este motivo, y con el objeto de evitar redundancia de información, se omiten sus listados y explicaciones en este informe, remitiéndose al lector al desarrollo previo.

2.2 Código Fuente Principal en C

El archivo `main.c`, presentado en el listado 2.1, implementa la lógica completa de este ejercicio, incluyendo el mapeo de registros, la configuración del reloj del sistema y los periféricos implicados.

```
1 #include <stdint.h>
2
3 typedef volatile uint32_t register_t;
4
5 #define GPIOA_BASE_ADDRESS 0x40010800
6 #define GPIOA_CRL (*(register_t *) (GPIOA_BASE_ADDRESS)) // GPIOA config register L (0-7)
7 #define GPIOA_CRH (*(register_t *) (GPIOA_BASE_ADDRESS + 0x04U)) // GPIOA config register H
8 #define GPIOA_ODR (*(register_t *) (GPIOA_BASE_ADDRESS + 0x0CU)) // GPIOA Output data register
9 #define GPIOA_BSRR (*(register_t *) (GPIOA_BASE_ADDRESS + 0x10U)) // GPIOA Set/Reset register (
10 #define GPIOA_BRR (*(register_t *) (GPIOA_BASE_ADDRESS + 0x14U)) // GPIOA Reset register (atomic)
```

```

11
12 #define GPIOC_BASE_ADDRESS 0x40011000
13 #define GPIOC_CRL      (*(register_t *) (GPIOC_BASE_ADDRESS))           // GPIOC config register L (0-7)
14 #define GPIOC_CRH      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x04U))   // GPIOC config register H
15                               (8-15)
16 #define GPIOC_ODR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x0CU))   // GPIOC Output data register
17 #define GPIOC_BSRR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x10U))  // GPIOC Set/Reset register (
18                               atomic)
19 #define GPIOC_BRR      (*(register_t *) (GPIOC_BASE_ADDRESS + 0x14U))  // GPIOC Reset register (atomic)
20
21 #define RCC_BASE_ADDRESS 0x40021000
22 #define RCC_APB2ENR      (*(register_t *) (RCC_BASE_ADDRESS + 0x18U))   // Advanced Peripheral Bus 2
23                               clock enable register
24 #define RCC_CFGR      (*(register_t *) (RCC_BASE_ADDRESS + 0x04U))      // Clock configuration register
25
26 // Periférico AFIO (Alternate Function I/O)
27 #define AFIO_BASE_ADDRESS 0x40010000
28 #define AFIO_EXTICR4      (*(register_t *) (AFIO_BASE_ADDRESS + 0x14U)) // External interrupt
29                               configuration register 4
30
31 // Periférico EXTI (External Interrupt Controller)
32 #define EXTI_BASE_ADDRESS 0x40010400
33 #define EXTI_IMR      (*(register_t *) (EXTI_BASE_ADDRESS + 0x00U)) // Interrupt Mask Register
34 #define EXTI_FTSR      (*(register_t *) (EXTI_BASE_ADDRESS + 0x0CU)) // Falling Trigger Selection
35                               Register
36 #define EXTI_PR      (*(register_t *) (EXTI_BASE_ADDRESS + 0x14U)) // Pending Register
37
38 // Periférico NVIC (Cortex-M3 Internal Core Peripheral)
39 #define NVIC_ISER1      (*(register_t *) (0xE000E104U)) // Interrupt Set-Enable Register 1
40
41 // Periférico ADC1
42 #define ADC1_BASE_ADDRESS 0x40012400
43 #define ADC_SR      (*(register_t *) (ADC1_BASE_ADDRESS + 0x00U)) // Status Register
44 #define ADC_CR1      (*(register_t *) (ADC1_BASE_ADDRESS + 0x04U)) // Control Register 1
45 #define ADC_CR2      (*(register_t *) (ADC1_BASE_ADDRESS + 0x08U)) // Control Register 2
46 #define ADC_SMPR1      (*(register_t *) (ADC1_BASE_ADDRESS + 0x0CU)) // Sample Time Register 1 (Ch
47                               10-17)
48 #define ADC_SMPR2      (*(register_t *) (ADC1_BASE_ADDRESS + 0x10U)) // Sample Time Register 2 (Ch 0-9)
49 #define ADC_SQR3      (*(register_t *) (ADC1_BASE_ADDRESS + 0x34U)) // Regular Sequence Register 3
50 #define ADC_DT      (*(register_t *) (ADC1_BASE_ADDRESS + 0x4CU)) // Regular Data Register (ADC_DR)
51
52 // Variable global para indicar el modo de canal
53 // 0: Sensor de temperatura interno (Canal 16)
54 // 1: Potenciómetro en PA0 (Canal 0)
55 volatile uint8_t channel_mode = 0;
56
57 // función para encender un número específico de LED en PA1-PA5
58 void set_leds(uint8_t count) {
59     uint32_t mask = 0;
60     for (uint8_t i = 1; i <= 5; i++) {
61         if (i <= count) {
62             mask |= (1 << i);           // Enciende el LED en PAi
63         } else {
64             mask |= (1 << (i + 16)); // Apaga el LED en PAi
65         }
66     }
67     GPIOA_BSRR = mask;
68 }
69
70 int main() {
71     // 1. Habilitación del clock de los periféricos
72     // Enciende clocks para GPIOA (bit 2), GPIOC (bit 4), AFIO (bit 0) y ADC1 (bit 9)
73     RCC_APB2ENR |= (1 << 2) | (1 << 4) | (1 << 0) | (1 << 9);
74
75     // Configura prescaler del ADC a PCLK2 / 6 (ADCPRE bits 14-15 = 0b10)
76     // Requerido para mantener la frecuencia del ADC por debajo de 14 MHz
77     RCC_CFGR &= ~(0x3 << 14);

```

```

72  RCC_CFGR |= (0x2 << 14);
73
74  // 2. Configuración de puertos GPIO
75  // PC13 como salida push-pull (LED interno de parpadeo)
76  GPIOC_CRH &= (~((register_t) 0xF << 20));
77  GPIOC_CRH |= 0x3 << 20;
78
79  // PC15 como entrada pull-up (pulsador de interrupción)
80  GPIOC_CRH &= (~((register_t) 0xF << 28));
81  GPIOC_CRH |= 0x8 << 28;
82  GPIOC_ODR |= (1 << 15);
83
84  // PA0 como entrada analógica (potenciómetro) y PA1-PA5 como salidas push-pull 50MHz
85  GPIOA_CRL &= (~((register_t) 0xFFFFF));
86  GPIOA_CRL |= (0x33333 << 4) | (0x0 << 0);
87
88  // 3. Configuración del ADC1
89  ADC_SMPR1 |= (0x7 << 18); // Tiempo de muestreo máximo (239.5 ciclos) para canal 16 (temperatura
90  )
91  ADC_SMPR2 |= (0x7 << 0); // Tiempo de muestreo máximo (239.5 ciclos) para canal 0 (potenció
92  metro)
93
94  ADC_CR2 |= (1 << 23); // Habilita el sensor de temperatura y referencia de voltaje interno (
95  TSVREFE)
96  ADC_CR2 |= (7 << 17) | (1 << 20); // Selecciona SWSTART como disparador y habilita el modo de
97  trigger externo
98  ADC_CR2 |= (1 << 0); // Enciende el periférico ADC (ADON)
99
100 // Calibración del ADC
101 for (volatile int i = 0; i < 1000; i++); // Espera para estabilizar el encendido del ADC
102 ADC_CR2 |= (1 << 2); // Inicia calibración (CAL)
103 while (ADC_CR2 & (1 << 2)); // Espera fin de calibración
104
105 // 4. Configuración de interrupción externa (botón en PC15)
106 AFIO_EXTICR4 |= 0x2 << 12; // Mapea línea EXTI15 a puerto C
107 EXTI_FTSR |= 0x1 << 15; // Disparo por flanco de bajada (al presionar botón)
108 EXTI_IMR |= 0x1 << 15; // Desenmascara interrupción de línea 15
109 NVIC_ISER1 |= 0x1 << 8; // Habilita interrupción EXTI15_10 en el NVIC (IRQ 40)
110
111 uint32_t toggle_counter = 0;
112
113 while (1) {
114     // Determina el canal activo según la selección del botón
115     uint32_t channel = (channel_mode == 0) ? 16 : 0;
116     ADC_SQR3 = channel;
117
118     // Inicia conversión por software
119     ADC_CR2 |= (1 << 22); // SWSTART
120
121     // Espera fin de conversión (EOC)
122     while (!(ADC_SR & (1 << 1)));
123
124     // Lee el resultado
125     uint32_t adc_val = ADC_DT & 0xFFFF;
126
127     if (channel_mode == 0) {
128         // Cálculo de temperatura en décimas de grado Celsius
129         // T = ((V25 - Vsense) / Avg_Slope) + 25
130         int32_t vsense = (adc_val * 3300) / 4095; // en mV
131         int32_t temp = ((1430 - vsense) * 100) / 43 + 250; // temp en décimas de grado
132
133         // Mapea la temperatura a los 5 LEDs
134         if (temp < 260) set_leds(1); // < 26 °C
135         else if (temp < 300) set_leds(2); // 26 - 30 °C
136         else if (temp < 340) set_leds(3); // 30 - 34 °C
137         else if (temp < 380) set_leds(4); // 34 - 38 °C
138         else set_leds(5); // >= 38 °C
139     }
140 }

```

```
135     } else {  
136         // Mapea el valor del potenciómetro (0 - 4095) a los 5 LEDs  
137         if (adc_val < 800) set_leds(1);  
138         else if (adc_val < 1600) set_leds(2);  
139         else if (adc_val < 2400) set_leds(3);  
140         else if (adc_val < 3200) set_leds(4);  
141         else set_leds(5);  
142     }  
143  
144     // Parpadeo periódico del LED interno en PC13  
145     toggle_counter++;  
146     if (toggle_counter >= 50) {  
147         GPIOC_ODR ^= (1 << 13);  
148         toggle_counter = 0;  
149     }  
150  
151     // Pequeño retardo entre lecturas  
152     for (volatile int i = 0; i < 10000; i++);  
153 }  
154  
155 return 0;  
156 }  
157  
158 void EXTI15_10_IRQHandler(void) {  
159     if (EXTI_IMR & (1 << 15)) {  
160         // Alterna el modo de canal (0: sensor interno, 1: potenciómetro)  
161         channel_mode = !channel_mode;  
162  
163         // Limpia bandera pendiente de interrupción en el EXTI  
164         EXTI_PR = (1 << 15);  
165     }  
166 }
```

Listing 2.1: Código fuente principal e implementación de conversión ADC (main.c).

2.2.1 Mapeo de Registros Adicionales

Con respecto al ejercicio anterior, se incorporan las constantes simbólicas necesarias para acceder a los siguientes periféricos y registros:

- GPIOA: Dirección base 0x40010800. Se definen los registros GPIOA_CRL para la configuración de pines bajos (0-7), y GPIOA_BSRR para modificar de forma atómica sus estados de salida.
- RCC_CFGR: Registro de configuración de reloj en la dirección 0x40021004, utilizado para establecer el factor de división del reloj del ADC.
- ADC1: Dirección base 0x40012400. Se accede a ADC_SR (registro de estado), ADC_CR1 y ADC_CR2 (registros de control), ADC_SMPR1 y ADC_SMPR2 (tiempos de muestreo), ADC_SQR3 (secuencia de conversión regular) y ADC_DT (registro de datos).

2.2.2 Configuración de Relojes y Prescaler del ADC

La frecuencia de reloj del periférico ADC en la arquitectura STM32F103C8T6 no debe superar el valor de 14 MHz para garantizar la precisión de las conversiones [Stm].

Para cumplir con este requisito, se accede al registro RCC_CFGR y se configuran los bits de división del reloj del ADC (ADCPRE, bits 14-15) escribiendo la constante binaria 0b10. Esto

aplica un factor de división por 6 sobre la frecuencia de reloj del bus APB2. Con un oscilador interno de 8 MHz, la frecuencia de trabajo del ADC se establece en 1.33 MHz, cumpliendo de forma holgada el límite del fabricante.

2.2.3 Configuración de los Puertos GPIO

Para posibilitar la interfaz física y el control del ADC1, se requiere la configuración de diversos pines de entrada y salida de propósito general. La distribución de estas conexiones en el laboratorio sobre la protoboard se ilustra en la figura 2.1, y consta de los siguientes periféricos:

- Leds de barra (Salidas): Los pines PA1, PA2, PA3, PA4 y PA5 se configuran como salidas digitales push-pull con una velocidad máxima de conmutación de 50 MHz. Esto se logra escribiendo el valor de control 0x3 en sus respectivos campos del registro GPIOA_CRL.
- Potenciómetro (Entrada analógica): El pin PA0 se configura en modo de entrada analógica escribiendo la constante 0x0 en su campo correspondiente en GPIOA_CRL. De esta forma, el circuito integrado de entrada digital se deshabilita para evitar consumos adicionales y asegurar que el nivel analógico de tensión llegue directamente al ADC1 sin perturbaciones.
- Botón de interrupción (Entrada digital): El pin PC15 se configura como entrada digital con resistencia interna de pull-up. Esto se realiza escribiendo la constante 0x8 en el campo correspondiente en el registro GPIOC_CRH y estableciendo en '1' el bit 15 de GPIOC_ODR.
- Led de parpadeo interno (Salida): El pin PC13 se configura en modo salida push-pull a 50 MHz escribiendo 0x3 en su campo de GPIOC_CRH.

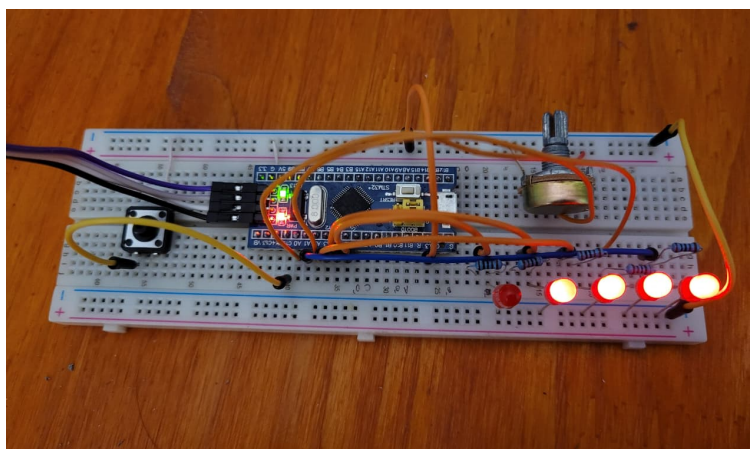


Figura 2.1: Disposición del circuito en protoboard para el Ejercicio Práctico N° 2.

2.2.4 Inicialización y Calibración del ADC

Para poner en marcha el ADC de forma precisa, se siguen los siguientes pasos en la función `main()`:

1. Tiempos de muestreo: El sensor de temperatura interno requiere un tiempo de muestreo mínimo de $17.1\mu s$ [Stm] para cargar de forma correcta la capacitancia del circuito interno de retención. Se escribe `0x7` en los bits 18-20 del registro `ADC_SMPR1` para configurar el máximo tiempo de muestreo disponible (239.5 ciclos de reloj de ADC). Del mismo modo, se configura el canal 0 en `ADC_SMPR2` con 239.5 ciclos para la señal del potenciómetro externo.
2. Activación de sensores internos y alimentación: Se escribe un '1' en el bit 23 (`TSVREFE`) del registro `ADC_CR2` para encender el sensor de temperatura. Se configuran los bits 17-19 de `ADC_CR2` como `0b111` (`EXTSEL = SWSTART`) y el bit 20 como '1' (`EXTTRIG = 1`) para habilitar el inicio de la conversión por software a través del evento `SWSTART`. Posteriormente, se escribe un '1' en el bit 0 (`ADON`) de `ADC_CR2` para alimentar el periférico.
3. Calibración: Se genera un retardo por software corto para esperar la estabilización del circuito de conversión analógico. Luego, se escribe un '1' en el bit 2 (`CAL`) de `ADC_CR2`. El hardware del STM32 ejecuta de forma automática una rutina para compensar los errores de desviación (offset) internos, y limpia el bit cuando el proceso finaliza. El código realiza una espera activa sobre este bit.

2.2.5 Lógica de Conversión por Muestreo (Polling)

Dentro del bucle de ejecución principal `while(1)`, se realiza el siguiente ciclo para procesar la conversión:

1. Selección de Canal: Se verifica la variable global `channel_mode` gestionada por la ISR. Si su valor es 0, se escribe el valor 16 (canal de temperatura) en el registro de secuencia `ADC_SQR3`. Si es 1, se escribe el valor 0 (canal externo del potenciómetro).
2. Disparo de Conversión: Se escribe un '1' en el bit 22 (`SWSTART`) de `ADC_CR2` para iniciar la adquisición de la muestra.
3. Espera de Fin de Conversión (EOC): El firmware permanece en un bucle `while` esperando que el hardware establezca en '1' el bit 1 del registro de estado `ADC_SR` (EOC, End of Conversion).
4. Lectura de Datos: Una vez finalizada la conversión, se lee el registro `ADC_DT` y se aplica una máscara para obtener la palabra de 12 bits resultante (rango de 0 a 4095).

2.2.6 Procesamiento de Datos y Visualización

Si el canal activo es el potenciómetro externo (valores de 0 a 4095), el valor digital se divide en cinco rangos equitativos mediante condicionales lógicos para activar una cantidad proporcional de LEDs mediante la función `set_leds(count)`.

Si el canal activo es el sensor de temperatura interno, el firmware realiza el cálculo de la temperatura empleando aritmética entera para evitar el uso de funciones de punto flotante de alto costo en microcontroladores. La tensión medida en milivoltios se determina como:

$$V_{SENSE} = \frac{ADC_{VAL} \times 3300 \text{ mV}}{4095} \quad (2.1)$$

Posteriormente, utilizando los parámetros provistos por la hoja de datos ($V_{25} = 1.43 \text{ V}$ y pendiente típica de $4.3 \text{ mV}/^{\circ}\text{C}$), se calcula la temperatura en décimas de grado Celsius mediante la expresión:

$$T^{\circ}\text{C} = \frac{1430 \text{ mV} - V_{SENSE}}{4.3 \text{ mV}/^{\circ}\text{C}} + 25^{\circ}\text{C} \quad (2.2)$$

El valor calculado se compara para activar la barra de LEDs con una escala adaptada al rango normal de funcionamiento de la placa de desarrollo (desde menos de 26°C hasta más de 38°C).

2.3 Relación entre Archivos de Desarrollo

La interacción del firmware se define de la siguiente manera:

1. La distribución del espacio de memoria establecida en `linker.ld` ubica el vector de interrupción externa apuntando a la dirección física del manejador `EXTI15_10_IRQHandler`.
2. Al presionar el pulsador externo en PC15, se genera un flanco descendente que dispara la interrupción. El procesamiento de la ISR en `main.c` alterna el valor de la variable `channel_mode` y limpia la bandera de interrupción en `EXTI_PR`.
3. El bucle principal lee de forma continua la variable `channel_mode` y conmuta el registro `ADC_SQR3` para iniciar la conversión en el canal correspondiente.
4. Los resultados de la conversión modifican los estados del puerto `GPIOA` a través de la función `set_leds`, variando la visualización en la barra de 5 LEDs de forma asíncrona a la pulsación.

3 Conclusiones

Tras el análisis, diseño e implementación del Ejercicio Práctico N° 2, se concluye que se han cumplido con éxito los objetivos propuestos. La configuración del periférico convertidor analógico-digital (ADC1) en un entorno Bare Metal ha permitido interactuar de manera directa con las señales analógicas del microcontrolador y procesar sus valores digitales equivalentes.

El ejercicio demuestra la viabilidad de implementar sistemas de adquisición híbridos, donde la conversión de datos se realiza mediante muestreo activo (polling del bit EOC) para garantizar lecturas continuas e instantáneas, mientras que la conmutación de los canales de lectura se resuelve de manera asíncrona a través de una interrupción externa desencadenada por hardware (línea EXTI15). Esta combinación optimiza el tiempo de CPU al delegar la detección de eventos de control al controlador de interrupciones (NVIC).

Por otro lado, la utilización del sensor de temperatura interno resalta la importancia de los periféricos embebidos para el diagnóstico y seguridad del chip. El procesamiento de sus valores mediante aproximaciones matemáticas basadas en aritmética entera demuestra ser una práctica de optimización crucial en firmware para sistemas embebidos de recursos limitados, evitando la inclusión de costosas bibliotecas de punto flotante.

Finalmente, el diseño de la interfaz visual mediante una escala en la barra de 5 LEDs externos permite comprobar de forma empírica y en tiempo real las variaciones de temperatura y el comportamiento lineal del potenciómetro de entrada. Este desarrollo proporciona los fundamentos de adquisición analógica necesarios para abordar implementaciones más complejas en etapas posteriores de este trabajo práctico.

4 Fuentes

- [Cor] *Cortex-M3 Technical Reference Manual*. r2p1. ARM Limited. 2010.
- [Stm] *RM0008 Reference manual: STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced ARM-based 32-bit MCUs*. Rev 21. STMicroelectronics. 2021.